

Invocation-Level Reliability of Tool Use in Language Model Agents

Working draft. Sections marked contain the exact structure to fill in once `scripts/run_real_suite.py` has been run against real models; everything else is complete and ready for review/editing.

Abstract

All figures below are measured on the frozen dataset. None is projected. Per-model sample sizes and the conditions not executed are stated in the Data Collection Note.

Tool-augmented language model agents are increasingly deployed to act on real systems, where a single incorrect function call can silently derail an entire task. Existing benchmarks largely report a single end-to-end or aggregate success score, which conflates two distinct failure channels — choosing the wrong tool and forming wrong arguments — and obscures how errors made early in a multi-step task propagate to corrupt everything downstream. We introduce a correct-invocation rate that disaggregates tool selection from argument correctness, measured both under a clean, teacher-forced context and under a model's own free-running context, across six open-weight models on a fixed, contamination-free suite of multi-step tasks with dependency depths from 1 to 8.

Separating the two arms lets us split degradation into a context-length component and a propagation component. On the 7–8B models the split is stark: the teacher-forced baseline declines gently with depth ($0.700 \rightarrow 0.543$) while the free-running rate collapses ($0.700 \rightarrow 0.174$), so that by depth 6 roughly **70% of a model's own clean-context capability is lost to its own earlier mistakes** (net propagation loss $L_t = 0.686$ and 0.684 , 89% CI widths under 0.16).

Our main result, however, is about the measurement rather than the models. **Under exact-match scoring against a fixed gold trajectory, both parameters of any such propagation model are determined by the scoring rule rather than by the data.** Severity is forced to its boundary -- 0 of 869 poisoned-context steps were correct, in every one of the four models that produced any (the fifth never left a clean context, leaving its π undefined rather than 1) -- and recovery is structurally unobservable: 0 of 580 poisoned steps returned to an on-track trajectory, against an expected 0.0028 coincidental returns. Both follow from one mechanism: a diverged chain holds a value that is not the gold one, and the gold value is the output of a tool whose constants the model never sees, so it is information the model has never received and cannot derive. A model that self-corrects often and one that never does produce identical observable data. Substituting the two forced values leaves $g_t = c_t p_t$ with **no free parameters**, so the parametric layer is not merely non-identifiable here but an identity in the measured per-step rates — and the fit, run anyway, returns 0.92 and 0.73 for a quantity that is exactly 1.000. This applies to any benchmark scoring calls against a fixed gold trajectory, so any work fitting a self-correction or recovery parameter under such scoring is reporting a pinned parameter. We give the mechanism, the arithmetic, and a concrete scoring change -- credit a call that correctly continues from the value the model actually holds -- which we implement and apply retrospectively to the collected data at zero API cost, un-pinning severity to interior estimates of $+0.149$ [$+0.021$, $+0.268$] and $+0.316$ [$+0.066$, $+0.503$]. Because it re-scores cached completions, any group holding raw completions can apply it without re-running a single call.

Two negative results are reported rather than omitted. A hypothesised shift in error composition along the chain is **untestable on this task family**, not for lack of statistical power but because 389 of 389 clean-context errors are tool *selection* errors and the argument channel is empty where the composition is defined. And the larger models in the suite sit at ceiling on these tasks ($p_t \approx 1.000$), so scale contrasts remain unresolved. Our pipeline, task suite, and analysis code are released, together with the measured provider rate limits that constrain replication and are documented nowhere else.

1. Introduction

Large language models are increasingly deployed as agents that call external tools and APIs to act in the world, and the point at which they do so has become a common source of failure. A model can reason correctly about what needs to happen and still produce a call that fails, either by choosing the wrong tool or by filling its arguments incorrectly. Benchmarks that report a single aggregate success number do not say where a call went wrong, and they typically reflect clean, single-shot conditions rather than the multi-step, state-carrying tasks real agents are asked to perform.

This gap matters because the two failure modes have different causes and different fixes. A model that reliably picks the right tool but frequently mis-fills arguments needs different intervention than one that hallucinates tools outright. And a model whose per-call accuracy looks acceptable in isolation can still fail badly in a multi-step chain if an early mistake corrupts everything that depends on it — a failure mode that a single end-to-end score cannot localize.

We study tool-use reliability at the level of the individual invocation. We define a call as correct when it both selects the right tool and supplies valid, correct arguments, and we measure this rate two ways: once under a clean, teacher-forced context that isolates how reliability itself degrades as more context accumulates, and once under the model's own free-running context, where an earlier mistake can poison everything after it. The gap between these two measurements is a direct, interpretable signature of error propagation, distinct from the ordinary decline in accuracy that comes from longer context alone.

Contributions.

1. A disaggregated correct-invocation metric and an evaluation protocol (matched clean/free-running scoring, syntactic-vs-semantic error classification, and a routing-task design that lets *selection* errors, not only argument errors, propagate) for measuring tool-use reliability at the invocation level rather than only end-to-end.
2. A formal propagation model with an interpretable severity parameter and origin-specific recovery rates, together with a fit-free propagation-loss metric that remains informative when the parametric model is not fully identifiable.
3. **A negative result about a scoring regime, not about a model family:** under exact-match scoring against a fixed gold trajectory, both the severity and the recovery parameters of any such propagation model are determined by the scoring rule rather than by the data. Severity is forced to its boundary and recovery is structurally unobservable, for one shared reason — once a chain diverges, the gold value is information the model has never received and cannot derive. This applies to any benchmark that scores calls against a fixed gold trajectory, so any work fitting a self-correction or recovery parameter under exact-match scoring is reporting a pinned parameter. We give the mechanism, the arithmetic, and a concrete scoring change that would make both quantities estimable.
4. An empirical demonstration — first on a fully-controlled simulated suite with known ground truth, then on real models — showing where the parametric model succeeds, where it is structurally limited by exact-match scoring, and which metric to trust in each regime.
5. A released, tested pipeline (task generation, harness, scoring, aggregation, Bayesian fitting, and figure generation), together with the measured provider rate limits that constrain replication and are published nowhere else.

2. Related Work

Large language models are increasingly deployed as agents that call external tools and APIs to act in the world, and the point at which they do so has become a common source of failure. A model can reason correctly about what needs to happen and still produce a call that fails, either by choosing the wrong tool or by filling its arguments incorrectly. This section surveys how the field has studied that failure, what it has measured, and where the measurement remains incomplete.

2.1 Foundations

The idea of teaching a model when and how to call an API goes back to Toolformer (Schick et al., 2023), which learned in a self-supervised way which tool to use, at what point, and with what arguments. ReAct (Yao et al., 2023) established the reasoning-action-observation loop that most agent frameworks still follow, and Reflexion (Shinn et al., 2023) added a self-critique step that later reliability work would build on. Gorilla (Patil et al., 2023) was among the first to measure API-call correctness at scale and to treat hallucinated calls as a distinct error to be counted rather than an occasional nuisance. Together these define the pipeline the rest of the literature examines: pick a tool, then form its arguments.

2.2 Benchmarks and what they measure

Most large tool-use benchmarks evaluate whether a task ultimately succeeds rather than whether each individual call was correct. ToolLLM (Qin et al., 2024) introduced ToolBench, built on thousands of real APIs, but the instability of those live endpoints made results hard to reproduce, which StableToolBench (Guo et al., 2024) addressed by simulating the APIs and caching responses. API-Bank (Li et al., 2023) scored calls by execution match across a smaller, controlled set of tools. The Berkeley Function Calling Leaderboard (Patil, Yan et al., 2025) has become the de facto standard, combining abstract-syntax-tree matching with executable checks and expanding over successive versions from single calls to parallel, multi-

turn, and multi-step settings. tau-bench (Yao et al., 2024) took a different route, grading the final state of a simulated environment after an agent-user dialogue and reporting reliability across repeated trials, where even strong models succeed on fewer than half of tasks and behave inconsistently from run to run.

The common thread is that these benchmarks report an aggregate number. That number is useful for ranking models, but it does not say where a call went wrong, and it usually reflects clean conditions in which the tools behave as expected.

2.3 Localizing the failure

A smaller group of benchmarks breaks the call apart. RoTBench (Ye et al., 2024) evaluates tool selection, parameter identification, and content filling as separate stages, and does so under increasing levels of noise, showing that accuracy that looks solid in clean conditions can fall sharply once the environment is perturbed. MTU-Bench (Wang et al., 2025) reports tool-selection and parameter accuracy directly and computes them without relying on a language model as judge, which makes the scores cheaper and more consistent. FuncBenchGen (Maekawa et al., 2025) generates contamination-free multi-step tasks as dependency graphs and finds that capable models often produce syntactically valid calls while carrying incorrect or stale argument values from one step to the next, with performance degrading as the chain of dependencies grows. This work is the closest in spirit to an invocation-level view, though it still reports task success rather than a single disaggregated measure of call correctness.

2.4 How the field categorizes errors

Across these studies the failures sort into two families:

- **Selection errors:** choosing the wrong tool, inventing a tool that does not exist, calling a tool when none was needed, or skipping a call that was required.
- **Argument errors:** malformed or unparseable output, missing required parameters, wrong types, wrong values, and fabricated arguments supplied instead of asking the user for the missing information.

Relign (Xu et al., 2025) formalizes this split into tool-selection and tool-usage hallucination and adds the option for a model to defer or ask for clarification rather than guess.

2.5 Error propagation in multi-step chains

Selection and argument errors are stateless: they describe a single call in isolation. In a multi-step task, where the arguments of one call are drawn from the outputs of earlier calls, a further structure appears that the two families do not capture. A single mistake at step t does not simply lower the score for that step; it changes the inputs every downstream step receives, so later calls can be locally well-formed and still globally wrong. This is the behaviour FuncBenchGen (Maekawa et al., 2025) isolates when it reports models carrying stale or incorrect values forward.

The distinction that makes this tractable is between two notions of a correct invocation. A call is *locally correct* if, given the correct inputs, the model selects the right tool and forms valid, correct arguments. A call is *globally correct* if it is correct inside the model's own run, where its inputs may already be poisoned by an upstream error. A model can be strong locally and still lose accuracy across a chain if it cannot recover once the context is corrupted.

2.6 Methods for improving reliability

The mitigations below share a target, and it is not the one this paper is about: they address how *often* an incorrect call is produced, whether by improving single-turn accuracy or by constraining output format. None of them speaks to what happens to a chain once an incorrect call has entered it. That is the gap the propagation framing addresses.

Proposed fixes sit at different points in the pipeline. Reliability alignment (Relign) trains models to abstain or clarify when they are unsure. Data-centric approaches such as ToolACE (Liu et al., 2025) and Hammer (Lin et al., 2024) improve function-calling accuracy through synthesized training data and by masking function and parameter names so models generalize beyond surface patterns rather than memorizing them. Constrained or schema-guided decoding removes formatting errors by construction, but it does not correct a well-formed call that carries the wrong value. Self-reflection and retry loops attempt to catch errors after execution, and a more recent line of work, including the paper anchoring this project (Healy et al., 2026), detects bad tool selection from a model's internal representations at inference time rather than trying to prevent it during training.

2.7 Metrics and the gap this study addresses

Evaluation across this literature uses exact match on the full call, tool-selection accuracy, parameter-level accuracy, abstract-syntax-tree matching, and success across repeated trials. A recurring observation is that tool-selection accuracy alone is necessary but not sufficient: a model that picks the right tool but passes a wrong argument scores well on selection and still fails the user.

Three gaps follow from this review. First, headline benchmarks aggregate their results and rarely localize where invocation fails, and the studies that do disaggregate tend to cover few models or non-fixed task sets. Second, cross-model comparisons that separate selection errors from argument errors are uncommon, and they are reported on different tasks with different metrics, so no clean and comparable picture exists. Third, most evaluation reflects a happy path, while the work that stresses models with noise, distractors, and dependency depth shows reliability dropping in ways a clean score hides. This work addresses all three: a disaggregated, depth-controlled, multi-model measurement with a fixed task suite and a single comparable metric.

3. Method

3.1 Definitions

A task is a sequence of calls $c_1, \dots, c_n$, where the arguments of c_t may depend on the outputs of earlier calls. Dependency depth is the length of the longest such chain. We define:

- **Correct invocation:** a call that both selects the correct tool and supplies valid, correct arguments.
- **Local correctness (p_t):** the probability step t is correct given a correct upstream history of length $t - 1$, measured by teacher-forcing the clean history at its true length. Because this baseline is measured at every depth rather than assumed constant, it captures context-length degradation (a model's accuracy naturally declining as the prompt grows) on its own, separately from propagation.
- **Global correctness (g_t):** the probability step t is correct inside the model's own free-running trajectory, where upstream inputs may already be poisoned.
- **Net propagation loss:** $L_t = 1 - g_t/p_t$, the fit-free, always computable signature of propagation: the fraction of reliability lost at depth t that is not explained by the context-length baseline alone.

3.2 A propagation model with recovery

We additionally model propagation as a three-state Markov process over a task's context: clean, poisoned-by-syntax, or poisoned-by-semantics, with transition probabilities driven by the measured baseline p_t , the measured syntactic error share $f_{\text{syn}}(t)$, and two free parameters — recovery rates r_{syn} and r_{sem} specific to each corruption origin, since syntactic failures are caught by the tool executor (which can return a structured exception) while semantic failures execute silently and have no natural signal to prompt recovery. A single severity parameter π scales the correctness probability while poisoned relative to the clean baseline: $\rho_t = (1 - \pi)p_t$. Occupancies evolve as

$$c_{t+1} = c_t p_t + s_t r_{\text{syn}} + m_t r_{\text{sem}} \quad s_{t+1} = c_t (1 - p_t) f_{\text{syn}}(t) + s_t (1 - r_{\text{syn}}) \quad m_{t+1} = c_t (1 - p_t) (1 - f_{\text{syn}}(t)) + m_t (1 - r_{\text{sem}})$$

with observed global correctness $g_t = p_t (1 - \pi x_t)$, where $x_t = s_t + m_t$ is the poisoned occupancy. Setting $r_{\text{syn}} = r_{\text{sem}} = 0$ recovers a simple absorbing-poison model; each is a special case of the general model, which we present to make the reviewer-facing progression of the framework explicit rather than only stating the final form.

3.3 Estimation

We fit $\pi, r_{\text{syn}}, r_{\text{sem}}$ with a Bayesian hierarchical model (PyMC, NUTS), pooling partially across model families and scales so that a model which collapses to its floor at shallow depth borrows statistical strength from related models rather than producing an unidentified point estimate. We report standard MCMC diagnostics (R-hat, effective sample size, divergence count) for every fit, and — critically — the posterior correlation between π and each recovery rate per model, which is the direct diagnostic for whether severity and recovery are separably identified from the data at hand (Section 5).

3.4 Task design

Two task sources are used, for internal control and external validity respectively.

Synthetic, contamination-free tasks. Each task is a hidden dependency graph over deterministic, executable functions; solving it means traversing the graph and feeding each function the outputs of its predecessors. This gives exact ground truth for every call and direct control of dependency depth (we use depths 1, 2, 4, 6, 8) and distractor density. Two task variants are used:

- *Linear tasks*, where the tool order is announced up front, so only argument-value errors can propagate.
- *Routing tasks*, where the correct next tool is a deterministic function of the incoming value rather than pre-announced, so a wrong tool choice also changes what is correct downstream. This is necessary because scoring tool selection against a fixed gold trajectory is only valid when the trajectory does not depend on the (possibly corrupted) value the model is holding; on routing tasks we score selection *conditionally* — against the tool that is correct given the ref the model actually holds — while separately recording divergence from the gold trajectory, so that an agent applying its own rule correctly to a poisoned input is not misclassified as a selection failure.

****Real benchmark tasks.** ** A held-out multi-step subset of [BFCL multi-turn/multi-step splits and/or tau-bench], scored under the identical local/global protocol, to check whether findings from the controlled synthetic suite generalize to real-world tool schemas and argument types.

3.5 Evaluation protocol

For each task and model, we: (1) present tool schemas and the task; (2) score selection (conditionally, per Section 3.4) and arguments strictly (exact match, after coercing digit-only JSON strings to integers so that a provider's number formatting is not scored as a value error). A soft score is computed but is identical to the strict score on this suite, since every argument is a single integer — see Section 7; (3) classify each error as syntactic (parse or schema failure, caught by the executor) or semantic (executes but is wrong). The executor can return either a structured exception or an opaque failure; only the structured condition was run (Section 7); (4) measure p_t via teacher-forcing; (5) measure g_t via the model's own free-running trajectory; (6) measure recovery directly from the logs, separately by origin, as the rate at which a context poisoned syntactically or semantically returns to an on-track chain on the following step, where on-track means the value carried in equals the value the gold trajectory expects. The within-step retry-recovery rate is a distinct quantity and is reported separately rather than folded into r_{syn} . These measured rates centre the recovery priors in the fit. Both the teacher-forced and free-running conditions use the same within-step retry budget, so that p_t and g_t are scored under identical rules and their ratio is not an artifact of asymmetric retrying.

3.6 Model suite

The suite is designed to vary one factor at a time:

Axis	Isolates	Models
Scale within a family	Effect of parameter count	[e.g., Qwen2.5-Instruct 7B/14B/32B/72B]
Family at matched scale	Effect of architecture/training	[e.g., Qwen2.5-7B vs Llama-3.1-8B vs ...]
Function-calling tuning	Effect of tool-specific fine-tuning	[FC-tuned model vs its general instruct base]
Proprietary reference	An absolute-scale ceiling	[e.g., GPT-class, Claude-class, reduced subset]

Exact model identifiers, provider, and access date are pinned and logged per run for reproducibility (Section 3.7).

3.7 Reproducibility

All decoding is greedy (temperature 0) for the primary measurement. Every model response is cached by (model, calling mode, exact prompt), so repeated analysis costs nothing. Task generation uses fixed seeds. All code, the exact task suite, and analysis scripts are released; see Appendix A.

4. Pipeline Validation on a Controlled Simulated Suite

Before spending API budget on real models, we validated the full pipeline — task generation, harness, scoring, aggregation, and the hierarchical fit — on six simulated model policies with known, configured ground truth (severity, recovery rates, and clean-baseline decay), run through the actual harness rather than an analytic shortcut. This section reports that validation in full, both because it is a legitimate methodological check that a reviewer will ask for, and because it produced a substantive empirical finding (Section 4.3) that directly shapes how we report results on real models.

4.1 Setup

Six policies across two families ("A", "B"), each at three severity tiers (weak/mid/strong), with configured clean baseline, severity, syntactic error share, and origin-specific recovery rates (full configuration in Appendix B). Each policy ran through the harness at depths 1, 2, 4, 6, 8, 400 tasks per depth (200×2 seeds), on the **linear** task variant. (The routing variant is exercised by the stress-test suite but not by this validation run; an earlier draft of this section incorrectly claimed both. Section 4.4 explains why the distinction matters.)

4.2 Harness hardening and artifact elimination

Validation surfaced nine measurement artifacts. Seven are eliminable by construction, and we state those as the structural constraints the harness now enforces rather than as a list of repairs, because the constraint is the reusable object. Two are not, and we keep those as narrative for a reason given at the end: the code and the configuration were both correct, and the measurement was invalid anyway.

Constraints enforced by construction. Each is locked by a regression test.

1. **Retry budgets are normalised across run modes.** Teacher-forced and free runs receive identical retry allowances, so p_t and g_t differ only in whether the upstream history is correct and not in how many attempts each step was given.
2. **Selection errors must be able to propagate.** The linear task family announces the tool order, so a wrong value can corrupt arguments but never tool choice; the routing variant makes the correct tool a function of the carried value, which is the condition under which a selection error has downstream consequences at all.
3. **Every poisoned state has a reachable exit.** A syntax failure whose retry budget is exhausted must not leave the chain permanently absorbed by a mechanism unrelated to the severity parameter.
4. **Selection is scored conditionally as well as against gold.** An agent applying the routing rule correctly to an already-poisoned value is not committing a selection error, and scoring it as one attributes argument propagation to the selection channel (§3.4).
5. **Stalled chains are labelled explicitly.** A step that never executed leaves the carried value stale, which is a distinct corruption mode from a wrong-value semantic error and must not be readable as one downstream.
6. **Composition estimators are restricted to fresh errors on clean contexts.** Errors that are wrong purely because of upstream poisoning carry no information about which error type a model produces, and including them inflates the semantic share. Validated against known configuration: the recovered syntactic share tracks the configured one after the restriction and does not before it.
7. **Every reported rate carries an interval.** L_t is a ratio of two estimated rates, so its uncertainty is not either one's; task-level bootstrap intervals are computed because task-level resampling preserves within-chain correlation.

Two artifacts were not preventable by construction, and that is their value.

The first was a definitional error hidden by a task property. "Clean context" is a property of the value carried *in*, but the harness compared the carried value against the expected *argument*. On a copy-argument task those are equal by construction, so the comparison was correct and every test passed. It becomes wrong only on a task where the required argument is a function of the carried value rather than a copy -- and it was invisible until we built one. Had the transformed-argument variant shipped without re-deriving the definition, every clean step would have been marked poisoned and L_t would have been meaningless, with nothing failing.

The second was the simulated policy's recovery mechanism, which did exactly what it was configured to do: it recovered at the configured rate, and the log-based estimator correctly recovered that rate. But it recovered by reading the gold output directly, and no real model has that access. The recovery parameter was therefore never estimable from observable data in any version of this study, and the measured-transition priors of §4.3 were built from transitions only a simulator can produce.

Correct code, correct configuration, invalid measurement. So the section makes two points rather than one: most measurement artifacts are eliminable by construction and should be stated as constraints, and some are visible only once a task exists that can distinguish the definitions -- which is an argument for building such tasks deliberately rather than for greater care.

4.3 Findings

A methodological contribution, stated first because it supersedes what this section originally reported.

Earlier drafts of this work described the severity and recovery parameters as *weakly identified*, evidenced by correlated posteriors, and treated that as a limitation. That description was too generous to the fit. The correct statement, derived in §5.4 from the real run, is stronger and less comfortable:

Under exact-match scoring against a fixed gold trajectory, the severity parameter is forced to its boundary and the recovery parameters are unobservable. The recurrence collapses to an identity in the measured per-step rates, with no free parameters. Yet a naive fit of the full model returns posterior means of 0.92 and 0.73 -- **confident estimates of quantities the scoring regime had already determined.**

The practical warning is that nothing in the standard diagnostic toolkit catches this. Our sampler reported $\max \hat{R} \leq 1.002$, minimum ESS of 5,823, and zero divergences. Those diagnostics are working correctly: they certify that the posterior was *explored* faithfully, and they are silent on whether the data *constrained* it. A well-behaved chain exploring a likelihood that is flat in a parameter produces exactly the picture we saw -- tight intervals around a value the prior and the scoring rule chose between them. **Clean convergence diagnostics are not evidence of identification**, and on gold-trajectory agent benchmarks the two come apart systematically rather than occasionally.

The constructive half is the dual-metric protocol, which we propose as a blueprint rather than a workaround. Anchor primary claims on the fit-free L_t , which is a ratio of two directly measured rates and therefore cannot be pinned by the scoring rule. Treat parametric posteriors as diagnostic, reported alongside their identifiability evidence, and never as a headline. Add conditional-on-state scoring (§5.4a) wherever a claim about severity, recovery or self-correction is intended, since that is the only way those quantities acquire an interior range. The first two of these cost nothing; the third re-scores cached completions and needs no new inference.

What follows are the findings from the simulated suite itself.

L_t measures net propagation loss, and that is not the same thing as a severity ranking. Both facts belong here, next to the numbers, because reporting either alone misleads. On the routing suite at depth 8, L_t is 0.355 [0.322, 0.387] for the policy configured at $\pi = 0.70$, 0.197 [0.170, 0.226] at $\pi = 0.40$, 0.154 [0.128, 0.182] at $\pi = 0.45$, and 0.025 [0.010, 0.038] at $\pi = 0.22$ (89% paired-bootstrap intervals). Five of the six pairwise comparisons separate with non-overlapping intervals, and every pair whose configured π differs by at least 0.18 separates. The sixth pair — $\pi = 0.45$ against $\pi = 0.40$ — overlaps, and its point estimates are ordered against the configured π .

This is expected rather than anomalous, and it is the single most important thing to understand about the metric. $L_t = \pi \cdot x_t$ is a **product**: the severity of poisoning multiplied by the poisoned mass accumulated by depth t . Two models with similar π but different error-mix dynamics have different x_t , so their L_t can order against their π . The linear suite reproduces this independently across families, where the policy configured at $\pi = 0.65$ has a *higher* L_8 (0.383) than the one configured at $\pi = 0.70$ (0.330), the two families differing in syntactic share and recovery rates.

So L_t answers "how much reliability is actually lost along a chain of this depth" — which is the operationally meaningful question, and which it answers without any fitted parameter. It does not answer "which model is intrinsically more fragile per unit of corruption." Where models differ mainly in severity, the two coincide; where they differ in error composition, they need not. We report L_t as the headline for the first question and never read a ranking off it for the second.

A model-free lag check ($\Delta_1 = P(\text{step } t+1 \text{ correct} \mid t \text{ correct}) - P(\text{step } t+1 \text{ correct} \mid t \text{ wrong})$) is large and positive for every policy (0.59–0.68 on routing), independently confirming that propagation is real and detectable without any parametric assumption.

The parametric severity/recovery fit is not fully identifiable under exact-match scoring. MCMC diagnostics are healthy (max $\hat{R}$ 1.002, min ESS ≈ 3400 , zero divergences), yet the fitted π does not recover the configured values: absolute error is 0.064–0.313 on routing (mean 0.189) and 0.023–0.385 on linear (mean 0.167). The posterior correlation between π and at least one recovery rate is substantial for every model — 0.84–0.89 on routing, 0.22–0.74 on linear — which is the direct diagnostic signature of non-identifiability.

This finding was re-examined specifically because it might have been an artifact. Section 4.2's fixes made the recovery rates measurable from labelled transitions for the first time, and those measured rates now centre the recovery hyperpriors; the prior centres land within 0.02 of the configured family means. The natural hypothesis was that correct, informative priors would resolve the non-identifiability. **They did not.** Fixing the inputs did not change the correlation structure, which is what a structural rather than mis-specification account predicts, and it rules out the most attractive alternative explanation. The mechanism is that under exact-match scoring, a step can only be judged globally correct while its context

is poisoned if it exactly recovers the true value — a merely locally-consistent continuation of an already-wrong input essentially never coincidentally matches the fixed gold trajectory — which collapses much of the distinction the model draws between "how bad is poisoning" and "how fast do you recover," since both manifest through the same observable event. A secondary contributor is likely task design: the synthetic tasks use exact integer arithmetic, which has no graded notion of "close but not quite correct" that soft matching could give independent empirical content to π .

Implication for reporting on real models. We report L_t as the primary, always-trustworthy quantity throughout this paper. We report the fitted π /recovery posteriors as secondary and diagnostic, always alongside their identifiability correlation, following exactly the fallback the method was designed with. Section 3.4's real-benchmark arguments (which, unlike our integer chains, have genuine graded closeness) would test directly whether identifiability improves outside the synthetic setting. That test was not run: the real arm uses the same integer-argument tasks, so the question remains open and is listed in the Data Collection Note.

4.4 What this validation can and cannot establish

Validating on simulated policies has a boundary that we state explicitly, because we crossed it and were caught by it.

The simulated policy is invoked as `policy(task, step, ref, attempt)`: it is *handed* the reference value carried into the step as an argument, delivered out-of-band rather than read from the conversation. A real model receives only the message list. Those interfaces are not equivalent, and the difference is precisely the dependency this study measures.

Because of that, the validation reported above could not detect that the free-running loop was appending neither the model's own call nor the tool's result to the conversation. Every simulated policy behaved identically whether the history was complete or empty, since none of them read it. The validation suite reported a healthy pipeline while the run loop could not have worked on any real model; the defect surfaced in the first minute of real API traffic and had survived 24 stress-test configurations.

We therefore scope this section's claim precisely: **it establishes that the estimation and aggregation pipeline recovers configured parameters from labelled transitions. It does not establish that the prompt-construction and observation-passing path is correct**, and no simulation with an out-of-band state interface can establish that.

A simulator validates the channels it exercises, and silently certifies the ones it does not. This is the transferable contribution of the section, and we state it as a principle rather than as a list of incidents because it is *predictive*: it says in advance where to look, namely at every channel the real interface has that the mock supplies some other way.

The nine defects below are one finding, not nine. Each is the same failure: a channel the real interface has, which the simulator bypassed, and which therefore reported healthy no matter what it did. What makes the pattern worth stating as a principle, rather than as a lesson about being careful, is that **the last two are not coding errors at all**:

- Comparing the value carried *in* against the expected *argument* is **correct** on a copy-argument task, where the two are equal by construction. It becomes wrong only on a task that can distinguish them. The copy variant was not hiding a bug; it was masking a *definitional* error, and it took building a task with transformed arguments to expose it. Had we shipped that task without re-deriving the definition, every clean step would have been marked poisoned and L_t would have been meaningless -- with no test failing.
- The simulator's recovery mechanism did **exactly what it was configured to do**. It recovered at the configured rate, and the log-based estimator correctly recovered that rate. But it recovered by reading the gold output directly, and no real model has that access. So the recovery parameter was never estimable from observable data in any version of this study, and the measured-transition priors of §4.3 were derived from transitions only a simulator can produce.

Correct code, correct configuration, invalid measurement. That is why the principle has to be applied by asking what each channel *is*, rather than by looking for mistakes.

We also record how long this took to see. **This pipeline documented the same non-identifiability across three successive reworkings -- adding measured-transition priors, then per-step resolution, then paired resampling -- each time treating it as a statistical property of the fit, before identifying that the scoring regime had determined both parameters in advance (§5.4).** The diagnostic was in the output every time; the question was not.

Bypassed channel	What the simulator did instead	What it certified falsely
the conversation itself	received <code>ref</code> as a call argument	a free-running loop that appended neither its own call nor the tool result
task difficulty	erred by construction at a configured rate	a linear suite on which real models never err, so nothing propagates
run-mode separation	one policy object served both arms	a teacher-forced baseline poisoned by the free run, giving $L_1 = -0.113$ where it is 0 by construction
the tool namespace	emitted a tool name no task defines	a selection-error share of exactly 0.00 at every step index, making H4 unfalsifiable
branch selection	recovered the value but branched on the corrupted one	a configured recovery of 0.60 observable as 0.179
the observable error label	kept a hidden corruption origin the logs cannot see	recovery rates biased $\sim 2\times$ toward each other, and hence biased priors
task identity	reused ids across generator seeds	task-grouped statistics merging unrelated trajectories
the definition of a clean context	compared the value carried IN against the expected ARGUMENT	identical under a copy-argument task, silently wrong under any transform: every clean step would be marked poisoned and L_t made meaningless
recovery itself	recovered by reading the gold output out of band	a recovery channel that cannot exist for a real model, whose priors then informed a fit whose non-identifiability we attributed to statistics

The last two are the sharpest illustrations of the principle, because neither is a coding error. Comparing the carried value against the expected argument is *correct* on a copy-argument task and only becomes wrong on a task that can distinguish them: the copy variant was masking a definitional error, and it took building a task that could tell them apart to expose it. And the simulator's recovery was not a bug at all -- it did exactly what it was configured to do -- but it did so through a channel no real model has, which means the recovery parameters were never estimable from observable data in any version of this study. We had documented that non-identifiability across three separate reworkings before identifying its cause.

Three of the nine were invisible to *every* assertion the suite made, because the suite asserted on rates the simulator itself produced. Two practical rules follow. First, tests should assert on the artifact sent to the provider — the message list — rather than on backend behaviour, since behavioural assertions inherit the mock's blind spots by construction. Second, any quantity the analysis derives should be checked against a configured ground truth *through the full pipeline*, not against the generator that produced it: four of the seven above were found only once a measured value was compared with the value it was configured from, and the last one only because the same configuration measured 0.34 at one seed count and 0.10 at another.

After the defect was fixed, this entire validation run was regenerated and compared element-by-element with the pre-fix artifacts. The clean baseline, syntactic share, success counts, trial counts, Δ_1 and every bootstrap L_t interval were **identical**, for the structural reason above, so the findings in Sections 4.2–4.3 stand as reported. The identifiability finding also survives: posterior correlations moved from 0.18–0.74 to 0.15–0.76, which is sampling noise across a change of environment, not a change in structure.

A separate caveat, not about the defect: this validation used the linear task variant, and the real-model pilot subsequently established that linear tasks carry no propagation signal on actual models ($p_t = g_t = 1.000$, $L_t = 0$), because the tool order is announced and the argument is a verbatim copy of the prior observation. The simulated policies still err on linear tasks by construction, so the estimator check remains valid, but it was performed on a task family whose real-model counterpart is degenerate. The real experiment's primary suite is therefore the routing variant, in which a wrong value also sends the agent down a wrong branch and selection errors can propagate.

5. Results

Full tables, per-model achieved sample sizes, and the diagnostic checks behind every claim here are in `docs/RESULTS_real_run.md`. This section states the findings; that document shows the working.

5.0 What is collected, and what is not

The sweep is **incomplete** and the results below are read accordingly. Cost on this task family is roughly quadratic in dependency depth — the tool schema grows with depth and the whole transcript is re-sent at every step — and the binding constraint is an undocumented per-model daily token allowance (§3.7). One day of the eight-day window has been spent. Depths 1, 2 and 4 are complete for five of six models; depth 6 is partial; **depth 8 has no data on any model**, and one model (`gpt-oss-20b`) has none at all because pilot runs earlier the same day had consumed 199,814 of its 200,000 daily tokens before the sweep began.

Since L_t grows with depth, and since the larger models only begin to make errors at depth, the missing bins are disproportionately the informative ones. Nothing below should be read as final, and two of the hypotheses are explicitly unresolved for this reason rather than answered weakly.

Models run **nested prefixes** of one task suite, sized to their individual token allowances, so n differs across models by design; cross-model comparisons are made on the common prefix. §3.7 gives the argument and `tests/test_nesting.py` asserts the property it depends on.

5.1 Per-depth clean baseline and global correctness

The two arms separate cleanly on the two small models. On `llama-3.1-8b-instant`, the teacher-forced baseline p_t falls from 0.700 at depth 1 to 0.549 at depth 6 — a 22% relative decline, which is the context-length effect the design exists to isolate — while the free-running rate g_t falls from 0.700 to 0.167, a 76% decline. `allam-2-7b` behaves the same way (0.533 → 0.556 baseline, 0.533 → 0.167 free-running). Figure 1.

$L_1 = 0.000$ **exactly** on every model, not approximately. At depth 1 both arms construct an identical prompt and share one cached response, so the baseline-purity property holds by construction rather than within tolerance. It is the cheapest available check that the two arms are wired correctly.

The clearest evidence is not the depth curve but the **step-index profile**, which requires no model at all. On `llama-3.1-8b-instant` at depth 6, error counts per step index on the *same tasks* were 21, 32, 35, 37, 40, 40 in the free arm against 21, 17, 15, 18, 20, 20 teacher-forced. The arms are identical at step 0, as they must be since the context is the same there, then the free arm diverges upward and stays up while the teacher-forced arm stays flat. A flat teacher-forced profile is what rules out context growth as the explanation; 94 of 246 shared (task, step) cells disagree between arms.

The three larger models are at ceiling ($p_t = 0.865$ – 1.000 at every depth reached), so $L_t = 0$ there is a statement about the tasks, not about robustness. Their $p_t = g_t$ identity was flagged by an automated structural check and then diagnosed rather than trusted: on `llama-3.3-70b-versatile` every one of its seven errors falls at step 2 or 3 of a depth-4 chain, and step 3 is the *final* step, where no downstream call exists to poison. Propagation is impossible there by construction, not absent by failure, so the correct reading is **inconclusive** pending the deeper bins.

5.2 Net propagation loss

depth	llama-3.1-8b L_t [89% CI]	allam-2-7b L_t [89% CI]
1	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
2	0.145 [0.082, 0.222]	0.087 [0.022, 0.163]
4	0.391 [0.311, 0.474]	0.514 [0.429, 0.600]
6	0.686 [0.598, 0.795]	0.700 [0.619, 0.778]

L_t rises monotonically with depth on both models, and the intervals at adjacent depths do not overlap — the depth trend is resolved, not merely suggested. Figure 2.

The two models are **not** separated from each other: their intervals overlap at depths 4 and 6. That is consistent with §4.3's finding that L_t is a product $\pi \cdot x_t$ and therefore not a severity ranking; the fitted π values do differ (0.91 against 0.68, §5.4), while the net losses coincide. This is exactly the case the validation run predicted would arise, and it is reported here as a property of the metric rather than as a failure to distinguish the models.

Against the ceiling-level models the separation is total: $L_6 \approx 0.70$ against $L_t = 0.000$. But that contrast conflates severity with task difficulty and is not offered as a model comparison.

5.2a Where in a chain a model errs is a model property, and no aggregate exposes it

This section makes two claims. The first is about measurement: **an error on the final step of a chain cannot propagate**, since no downstream call exists for it to corrupt, so it lowers g_t by its own contribution and adds nothing to the poisoned mass. Any study measuring L_t at a fixed maximum depth therefore under-states propagation loss, most severely where chains are shortest. $L_1 = 0$ is not a separate sanity check that the metric behaves; it is the **limiting case** of this bias, where every error is terminal by construction.

The second claim is about models, and it is the more useful one. The terminal-error share is not a constant determined by depth -- it is a **property of the model**, because *where along a chain a model makes its errors* varies between models at equal accuracy. And **no aggregate metric exposes it**. Two models with identical per-call accuracy, one erring early and one erring late, produce different propagation losses and would be ranked differently by L_t while being equally accurate. The late-erring model looks robust and is not.

Measured terminal-error share with 89% intervals, against the $1/d$ that uniformly distributed errors would give:

model	depth	errors	terminal share	89% CI	uniform expectation	L_t
llama-3.1-8b-instant	1	18	1.000	--	1.000	0.000
	2	49	0.714	[0.586, 0.821]	0.500	0.145
	4	165	0.333	[0.274, 0.397]	0.250	0.391
	6	320	0.197	[0.162, 0.236]	0.167	0.686
allam-2-7b	2	78	0.641	[0.542, 0.732]	0.500	0.087
	6	152	0.191	[0.141, 0.249]	0.167	0.728
llama-3.3-70b-versatile	4	7	0.857	[0.488, 0.992]	0.250	0.000
	6	21	0.286	[0.136, 0.482]	0.167	+0.190

The pattern is carried by llama-3.1-8b-instant and allam-2-7b, on 49-320 errors per cell, where the terminal share sits consistently *above* the uniform expectation at every depth and the intervals are narrow enough to say so. That is the load-bearing evidence: both models err later in a chain than chance would predict, plausibly because context is longest there, and the effect is large at shallow depth (0.714 against 0.500) where it does the most damage to L_t .

llama-3.3-70b-versatile is the vivid case but the weak one, and we do not lean on it. Its depth-4 terminal share of 0.857 rests on 6 of 7 errors, with an 89% interval of [0.488, 0.992] -- wide enough that it is compatible with anything from "somewhat above uniform" to "essentially all terminal". What it does show unambiguously is the consequence: with 6 of its 7 errors terminal, L_4 came out at exactly 0.000, which reads as a model immune to propagation. At depth 6 the same model gives $L_6 = +0.190$ with an 89% interval of [+0.062, +0.345], which excludes zero. An automated check flagged the $p_t = g_t$ identity, we classified it as benign on exactly this ground rather than as suite degeneracy, and the next bin confirmed it.

Two notes on the status of these two cells. The depth-4 row is **final**: that model's depth-4 allocation is complete, so its 6-of-7 terminal share will not improve with further data, which is a further reason the general claim is carried by the two high- n models rather than by this one. The depth-6 row has since gained tasks and the estimate moved from +0.140 [+0.000, +0.362] to +0.190 [+0.062, +0.345] -- inside the earlier interval, so confirmatory rather than a revision, and now separated from zero.

Consequences for anyone measuring propagation. The reported loss is a lower bound whose shortfall depends on a model-specific quantity, so cross-model comparisons of L_t at a fixed depth are confounded by error position. Three mitigations, all cheap: report the terminal-error share alongside L_t so the size of the bias is visible; read the depth *trend* rather than any single depth, since the bias attenuates the trend rather than reversing it; and treat error position as a reportable model characteristic in its own right rather than as a nuisance parameter, because a model that fails late in long chains is a different engineering problem from one that fails early.

5.3 Model-free propagation check

Δ_1 is positive for every model that makes any errors at all: +0.556 (llama-3.1-8b-instant), +0.322 (allam-2-7b), +0.880 (llama-3.3-70b-versatile), +1.000 (gpt-oss-120b). qwen3.6-27b produced no errors, so Δ_1 is undefined for it. Figure 7. Propagation is therefore confirmed with no parametric assumption whatsoever.

But the check is weaker than it looks here, and §5.4 explains why: the second term of Δ_1 is **exactly zero for every model**, so Δ_1 reduces to $P(\text{ok} \mid \text{ok})$ and stops being an independent quantity. We report it because a positive Δ_1 was pre-registered as the model-free confirmation of propagation and it delivers that; we do not treat its magnitude as an independent severity measure, because on this task family it is not one.

5.4 The scoring regime, not the models, fixes both parametric quantities

This is the paper's central negative result, and it is a statement about a class of measurement rather than about the models we happened to measure.

Severity is forced to its boundary. π is defined so that correctness under a poisoned context is $(1 - \pi)p_t$, making π a ratio of two directly observable conditional rates that needs no fit at all. Measured: **0 of 869 poisoned-context steps were correct**, so $\pi = 1.000$.

Pooled zero and per-model zero are different claims, so we state both. Every model that produced any poisoned step has **exactly zero** poisoned-context successes:

model	poisoned steps	successes	95% upper bound on the rate
llama-3.1-8b-instant	510	0	0.0059
allam-2-7b	335	0	0.0089
llama-3.3-70b-versatile	16	0	0.171
gpt-oss-120b	8	0	0.312
qwen3.6-27b	0	--	π undefined

The support is uneven, and we describe it as such rather than as "four of five", which would imply more uniformity than exists. **Two models carry the empirical claim** (510 and 335 poisoned steps; 95% upper bounds 0.0059 and 0.0089). **Two are consistent but uninformative** -- at 8 and 9 poisoned steps, bounds of 0.31 and 0.28 exclude almost nothing and contribute almost no evidence. **One is undefined:** qwen3.6-27b never left a clean context, so it has no severity at all rather than a severity of zero, and it is not pooled into any suite-level figure. What extends the claim past the two informative models is the mechanism below, which implies the result for any model under this scoring rule.

Recovery is structurally unobservable. $P(\text{next correct} \mid \text{thiswrong}) = 0.000$, and 0 of 580 poisoned steps with a successor returned to an on-track context.

Both follow from one mechanism. A poisoned step holds a value that is not the gold one. Scoring the call correct requires emitting exactly the gold argument, and the gold value at step t is the output of a tool whose constants are never exposed to the model. After divergence that value is information the model has never received and cannot derive: the only route to it is coincidence, at **Math input error** per opportunity. Over the 284 recovery opportunities the expected number of coincidental returns is **0.0058**. So a model that self-corrects 20% of the time and one that never self-corrects produce *identical observable data*, and likewise a model that partially survives poisoning is indistinguishable from one that never does. Severity is pinned at 1 and recovery is unidentified for the same reason.

What that does to the parametric layer. Substituting $\pi = 1$ and $r_{\text{syn}} = r_{\text{sem}} = 0$ into the recurrence of §3.2 leaves

$$g_t = c_t p_t, \quad c_t = \prod_{j < t} p_j, \quad L_t = x_t = 1 - c_t$$

with **no free parameters at all**. The state model is not merely non-identifiable here; it is an identity in the measured per-step rates, and L_t reduces to "the probability the chain has already erred". The three-parameter fit is therefore redundant on this data — and worse, it is *misleading*: it reports posterior means of 0.92 and 0.73 for a quantity that is exactly 1.000, because it fits a smooth recurrence to depth-pooled aggregates under a prior centred at 0.5. We report the directly measured values as primary and the posteriors only to show the discrepancy.

Scope of the claim. Nothing above depends on which models were run, on the task family, or on the specific numbers. It depends only on scoring calls against a fixed gold trajectory whose values the model cannot reconstruct — which is what execution-match and AST-match benchmarks do. Any study fitting a severity, self-correction or recovery parameter under such scoring is reporting a parameter that its scoring rule has already determined. We did not notice this for three successive reworkings of this pipeline, each of which documented the resulting non-identifiability without naming its cause (§4.3, §4.4).

5.4a A concrete fix: implemented, tested, and applicable retrospectively

The remedy is a scoring change. It is small enough to specify precisely, cheap enough to implement, and -- because the response cache holds every raw completion -- we can apply it to data already collected and report what it does. **That last property is what makes it adoptable:** any group with cached completions can re-score without re-running a single API call. Ours cost 0 calls and 881 cache hits.

Operationally: credit a call when it *correctly continues from the value the model actually holds*, rather than when it matches the gold trajectory. A step carrying a wrong value forward, which applies the right tool and the right argument rule to that value, counts as correct-given-state; a step that mis-applies the rule counts as wrong whether or not its input was already corrupted.

Half of it already existed. Routing tasks were already scored on both `selection_matches_gold` (did it name the tool gold names) and `selection_correct` (did it name the tool correct *given the ref it holds*). The second is exactly conditional-on-state scoring, and it is why selection errors stayed measurable after divergence while argument errors did not. We added the argument-side counterpart, `args_correct_given_state` -- does the argument equal the required function of the held value, a verbatim copy on the copy variant and $(\text{held} + k) \bmod M$ on the transformed one -- and `correct_given_state` requiring both.

It un-pins severity. Estimates below are pooled across steps, **stratified by the parity of the held value** (see the confound in §5.4b) and bootstrapped over tasks:

model	gold-agreement π	conditional π , parity-stratified	89% CI
llama-3.1-8b-instant	1.000 (boundary)	+0.149	[+0.021, +0.268]
allam-2-7b	1.000 (boundary)	+0.316	[+0.066, +0.503]

Clean-context steps score identically under both rules (0.658 vs 0.658), as they must -- a clean step's held value is the gold value -- which is the correctness check on the implementation.

What this licenses, and what it does not. Severity moves from a boundary artifact to an interior estimate whose interval excludes zero. So the strong reading -- that propagation here is *purely* information loss with no competence cost -- is **not supported**: holding a wrong value does measurably degrade rule application. The supported reading is quantitative rather than categorical:

Most of the measured propagation loss is **information loss** rather than competence degradation. Gold-agreement scoring attributes all of it to severity ($\pi = 1$); conditional scoring attributes 0.15--0.32 of it to genuine degradation and the remainder to the gold trajectory having become unreachable. Both quantities are real; only the second is a property of the model.

We flag this as a hypothesis the data supports rather than an established result, for two reasons. The intervals are wide and rest on 269--397 poisoned steps per model with only two models contributing. And the *scope* is narrow in a way that matters: on the copy variant, a correct argument means transcribing an integer the model was shown one turn earlier. That a model reliably copies a visible number is a weak competence test, so `args_correct_given_state` = 1.000 among poisoned steps -- these models never mis-transcribe -- may not survive a task where argument construction is harder. **The transformed-argument condition (§5.6) is therefore a test of this decomposition's generality, not only an H4 fix:** there the argument requires arithmetic on the held value rather than a copy, so the argument channel can fail on competence grounds. If the decomposition holds there, it is promoted; if not, it stays suggestive with its limits visible.

The transformed condition also re-tests the parity effect of §5.4b, for a reason worth stating: under `arg_shift` the value held at each step is $(\text{previous output} + k) \bmod M$ rather than a raw tool output, so the parity distribution of held values need not match the copy variant's. If the effect persists there it generalises; if it vanishes, it was an artifact of one variant's value distribution. Either outcome is reportable and costs nothing extra, because that condition is already scheduled.

What it costs, stated plainly. There is no longer a single right answer per call: scoring must reconstruct what the model held at each step and evaluate a predicate against it, which is more implementation and more room to disagree about the predicate. Cross-system comparison gets looser, since two systems can both be self-consistent while doing different things. And a high correct-invocation rate stops implying end-task success -- a perfectly self-consistent agent working from a wrong value still fails the task. Exact-match scoring buys comparability at the price of making severity and recovery unmeasurable; conditional scoring makes the opposite trade. **Reporting both is the defensible option**, which is why we keep gold-agreement as the headline and argue the conditional variant is required for any claim about severity or recovery.

5.4b Rule-following is measurable directly, and aggregates hide it

Define **discrimination** as $P(\text{pick first-listed tool} \mid \text{ref even}) - P(\text{pick first-listed tool} \mid \text{ref odd})$. The routing rule text lists the even branch first, so 0 means the model ignores the value it is supposed to condition on, +1 means it applies the rule perfectly, and a negative value means it is *anti*-correlated with the rule.

model	<i>n</i>	first-listed, overall	ref even	ref odd	discrimination	<i>z</i>
qwen3.6-27b	298	0.497	1.000	0.000	+1.000	—
openai/gpt-oss-120b	358	0.492	0.983	0.006	+0.978	+87.9
llama-3.3-70b-versatile	160	0.481	0.880	0.052	+0.828	+18.9
llama-3.1-8b-instant	1,014	0.478	0.551	0.406	+0.146	+4.7
allam-2-7b	680	0.685	0.597	0.774	−0.176	−5.0

The aggregate is uninformative and the conditional statistic is decisive. Overall first-listed rates are 0.481–0.497 for four of five models: indistinguishable, and consistent with no position bias whatever. The *same data* resolves discrimination across the full range, and the resulting order matches the reliability order independently established in §5.1 -- qwen3.6-27b never errs, allam-2-7b has the largest L_t . This makes discrimination a per-model diagnostic of *whether a model performs the task at all*, distinct from how accurately it performs it, and it is invisible to any metric aggregated over the conditioning variable.

These are three qualitatively different relationships to the rule, not one spectrum from strong to weak.

Derives it. qwen3.6-27b scores +1.000 with zero errors across 298 free-arm steps, and emits an explicit <think> block computing the parity before every call. It is not approximating the rule; it evaluates it.

Weakly follows it. llama-3.1-8b-instant scores +0.146 at $z = +4.7$ -- clearly positive, so it does condition on the ref, but it gets the conditional wrong most of the time it matters.

Is anti-correlated with it. allam-2-7b scores **−0.176** at $z = −5.0$. This is not blindness to the conditioning variable: **its position bias is strongest exactly where that bias is wrong**. It picks the first-listed tool *more* often when the ref is odd (0.777) than when even (0.595), and its accuracy is 0.595 on even refs against 0.223 on odd. A model merely ignoring the rule would sit near 0 and score about 0.5 on both; a negative value means the ref is informative about its choice in the wrong direction. So its contribution to the suite's propagation loss is not "a weak model that errs more" -- it is a model whose tool selection is actively mis-conditioned.

Why discrimination and not accuracy. With a fixed presentation order the correct tool for an even ref *is* the first-listed tool, so "always picks the first-listed option" and "applies the rule but only succeeds on even refs" predict the same accuracy pattern; accuracy cannot separate them. Discrimination conditions on the ref rather than on the outcome, so a pure position bias returns 0 whatever its accuracy. A dedicated control that randomises presentation order per step removes the confound outright and is reported in §5.9; the four resulting cells (ref parity x presentation order) settle it without relying on the statistic at all.

A revision, recorded rather than absorbed. Earlier analysis of this effect used a cache replay covering depths 1, 2 and 4. The step records now carry the held value and presentation order directly, so the statistic above is computed over all collected data including depth 6, and two numbers moved. allam-2-7b's overall first-listed rate is 0.685 rather than ~0.80 and its discrimination −0.182 rather than ~0, so the qualitative conclusion strengthens while its characterisation as an *unconditional* position bias does not survive. More importantly, a claim we had made about llama-3.1-8b-instant did not survive, and we state the correction rather than only the corrected number:

On depths 1, 2 and 4 that model's rule-application accuracy was 0.525 on even refs against **0.742** on odd -- a large asymmetry, which we attributed to the model. On the complete data including depth 6 it is 0.562 against **0.588**. **The asymmetry was a property of the shallow subset, not of the model.** Its discrimination likewise fell from +0.267 to +0.146, though it remains clearly positive.

The narrower surviving claim is that parity strongly affects rule application in one model and weakly or not at all in the others. The earlier, broader version would have reached print had the statistic not been recomputed on complete data, which is the reason we report the trajectory of the number and not just its endpoint.

5.5 Severity and recovery: the fitted posteriors

Recovery is not observable under exact-match scoring, and that is a scoping statement about the method rather than a finding about the models. $P(\text{next call correct} \mid \text{current call wrong})$ is 0.000, and 0 of 580 poisoned-context steps with a following step returned to an on-track context.

We initially read this as "these models never recover". That reading is not supported by the data, and the reason is decisive. Returning to on-track under exact-match scoring requires emitting exactly the gold argument. The gold value at step t is the output of a tool whose constants are never exposed to the model, so once a chain diverges the gold value is information the model has never received and cannot derive. The only route back is coincidence, at **Math input error** per opportunity, giving an expected 0.0058 coincidental returns over the 284 observed. **A model that self-corrects 20% of the time and one that never self-corrects produce identical observable data**, because the self-correcting model still cannot emit a number it has never seen.

So r_{syn} and r_{sem} are **not identified** here; they are not measured as zero. The three-state recurrence of §3.2 reduces to its absorbing special case -- $x_t = 1 - \prod_{j < t} p_j$ and $L_t = \pi \cdot x_t$ exactly, with no recovery term -- as a consequence of the *scoring regime*, not of model behaviour.

This explains the identifiability failure across every version of this pipeline, including the simulated ones.

The simulated policies did recover, but they recovered by reading the gold output directly, i.e. by being handed the true value out of band -- exactly the mock-interface channel §4.4 is about. The recovery channel was therefore never estimable from observable data in any version of this study, and the measured-transition priors of §4.3 were derived from transitions only a simulator can produce. §4.3's finding that correct priors still could not resolve the π /recovery confound is not a statistical accident; it follows from the scoring regime, and we now name that as the cause.

Two consequences stand. First, Δ_1 degenerates: its second term is structurally zero, so it equals $P(\text{ok} \mid \text{ok})$ and confirms propagation without being an independent severity measure. Second, the *apparent* improvement in identifiability on real data -- largest π /recovery correlation 0.54 against 0.84--0.89 simulated -- must not be read as the fit working better. There is no recovery variation left for π to trade against, because recovery cannot be observed at all. A parameter that cannot move is not identified; it is pinned.

What would make recovery estimable. Scoring would have to credit a return to a *self-consistent* trajectory rather than the *gold* one -- a model that detects the inconsistency, restarts from the seed value stated in the prompt, and proceeds consistently. That is a scoring-regime change and it is the prerequisite for the recovery half of the state model to be estimable at all.

Zero syntactic errors. Every one of roughly 1,900 recorded invocations executed, and no retry ever fired. A syntactic error is by definition a call that fails to execute, so there were none to classify: $f_{\text{syn}} = 0$. The fitted r_{syn} values in Figure 3b are therefore prior, not posterior, and the figure is labelled as such. This was flagged by the same automated check that catches bucketing defects, and verified against raw rows before being accepted, because an identical symptom in the simulated suite *was* a defect.

Only two of five π estimates are informative: 0.91 [0.82, 0.98] for `llama-3.1-8b-instant` and 0.68 [0.55, 0.82] for `allam-2-7b`. The three ceiling-level models produce too few errors to update the prior, and their posteriors span almost the entire unit interval; reporting 0.49 for `gpt-oss-120b` would be reporting the prior mean back. MCMC diagnostics are healthy throughout (max R-hat 1.0014, min ESS 5,823, zero divergences), which says the posterior was explored properly and says nothing about whether the data constrained it.

5.6 Error-type composition along the chain (H4): withdrawn

H4 is untestable on this task family, and the reason is more informative than a power limitation. Of 302 clean-context errors, **302 are tool-selection errors and 0 are argument errors**. The selection-to-argument ratio the hypothesis is about is undefined at every step index: there is no mix, so there is no shift in the mix to detect. This was verified directly against raw rows for both models rather than inferred from the aggregate.

The mechanism is structural. On a routing task a clean-context failure means the parity rule was mis-applied -- the wrong tool was named. The argument, by contrast, is copied verbatim from the previous result stated in the observation, and these models essentially never get that wrong *while holding a correct value*. Argument errors do occur -- 145 of 289 poisoned-context errors name the gold tool while carrying a wrong value, because the rule applied to a corrupted value can coincidentally land on the right branch -- but those are propagated errors, which the composition deliberately excludes as not being fresh evidence about what kind of error the model makes.

Power was not the binding constraint, and we checked. Projected against the *measured* per-model accuracies, the allocation yields four contiguous usable step bins, enough for a rank correlation to reach $p < 0.05$ at $\rho = -1$. (An earlier projection of six bins assumed a uniform 0.50 accuracy across models; correcting it for the three ceiling-level models, which contribute almost no errors however many tasks they run, drops it to four.) H4 would have been marginally testable on power grounds. It fails because one of its two categories does not occur where the composition is defined.

What would be needed. A task variant whose arguments can be wrong *independently of tool choice* — arguments requiring a transformation of the carried value rather than a verbatim copy, or multi-argument calls where one field can be wrong while the tool is right. That is a task-design change, not a budget change, and we state it as the concrete prerequisite rather than as future work.

The per-step aggregation machinery, and the finding that per-depth pooling recovers only 26% of a configured composition span where per-step resolution recovers 75%, are retained from the validation run (§4) and remain the right way to test H4 on a task family where both categories occur.

Why H4 is reported pooled, on evidence rather than on an appeal to power. A policy configured with a **flat** selection share (`flat-strong` in the validation suite) returned a per-step composition trend of $\rho = -0.738$ at $p = 0.037$ -- a significant trend where the ground truth has none. That is a directly observed false positive on a known null. The conventional inference from thin cells is "power is low, so pool", and this supports something stronger and less comfortable: a test that manufactures a significant trend on a control at these counts is **unreliable in both directions**, so a per-model *non-significant* result at comparable n cannot be read as evidence of a true null either. The noise that invented a trend here can equally mask one elsewhere. We therefore report the **pooled suite-level trend as the primary claim**, give per-model ρ as **directional evidence only**, and interpret no single per-model result in isolation -- not because the cells are small, but because we watched the per-model test return a wrong answer on a case whose truth we knew.

What the transform-variant validation licenses, stated precisely. We ran the same four-policy ground-truth suite on the transformed-argument variant before spending any budget on it. It recovers the configured trend there ($\rho = -0.976$, $p < 0.0001$, flat control $\rho = +0.048$, $p = 0.91$), so the estimator is sound under either variant. But its output is **identical to the copy variant's, digit for digit**, because the simulated policy chooses selection-versus-argument from a configured share and the transformation changes only which number an offset is added to. The simulator is therefore blind to the very difference the transform condition exists to test.

Consequently this paper does **not** claim that H4 detectability was validated on the transform variant and the effect then confirmed on real models -- that phrasing would have the validation lending support it cannot lend. The two statements are separate: the estimator recovers a configured composition trend under either variant *in principle*; and whether real models produce argument errors when arguments require arithmetic rather than copying is established by the real run **alone**. We state this explicitly because the distinction is easy to elide.

This is the §4.4 principle recurring -- a simulator validates the channels it exercises and silently certifies the ones it does not -- with one difference worth recording: it was caught **before** the budget was spent, because the check was instrumented rather than assumed to clear. All eight earlier instances were found in data already collected. A validation whose verdict could have stopped the condition paid for itself here even though the verdict was "proceed".

5.7 Function-calling mode ablation

Native vs. uniform calling mode on a fixed subset, quantifying how much of the argument-error rate is a calling-mode artifact rather than a genuine model limitation. Report as a direct contrast per Methodology Section 6.

Scope note: this ablation covers five of the six models. `groq/allam-2-7b` rejects native tool calling outright ("tool calling is not supported with this model"), verified before the ablation was run rather than discovered during it. It is retained in the main uniform-mode sweep, where it is the 7B leg of the family-at-matched-scale axis against `llama-3.1-8b-instant`; dropping it would cost that axis entirely. Its absence from the calling-mode comparison means the ablation's five models span the two scale-within-family axes but not the smallest scale point.

5.8 Simulated vs. real: three divergences, real primary

The pre-registered rule makes real data primary on disagreement and treats the divergence itself as the object of analysis. There are three, and they are substantive rather than quantitative.

1. Recovery. Simulated 0.05–0.30 by configuration; real exactly 0. The simulated policies were built to have the recovery structure the state model assumes, so the validation exercised a channel that does not exist on real models here.

This is the clearest instance of the general lesson in §4.4 — a simulator validates the channels it exercises — showing up in the *parameters* rather than in the code.

2. Identifiability, and it moved in the direction the validation did not predict. Largest π /recovery posterior correlation: 0.84–0.89 simulated, **0.54 real**. §4.3 established that even correctly measured priors could not resolve the confound on simulated data; on real data the confound is largely absent, because consequence 1 removes the quantity π was trading against. The method's headline weakness is milder in the setting it was built for than in the setting built to test it.

3. Error composition. Simulated policies produced both error types by construction; real models produce one. At depth 4 the simulated L_t spanned [+0.042, +0.169] against a real span of [+0.000, +0.514] — the real suite is far more dispersed, because it contains both near-ceiling and heavily-degrading models, which no simulated tier was configured to be.

What replicates: the **shape** of the result. L_t rises monotonically with depth, $L_1 = 0$ holds exactly, the teacher-forced arm stays flat where the free arm degrades, and L_t fails to separate two models whose π differ while separating those whose π differ widely — all four were established on simulated data first and all four hold on real models. The estimator transferred; the parameter regime did not.

6. Discussion

6.1 What to do instead: the diagnosis generalises and the fix is cheap

The central result of this paper is a diagnosis with a remedy, and the two belong together. Separating them would leave either a complaint or an unmotivated proposal.

The diagnosis generalises. Any evaluation that scores a call against a fixed gold trajectory, and whose environment does not hand the model the gold intermediate values, forces the same two conclusions: a severity parameter pinned at its boundary and a recovery parameter that no amount of data can identify. The argument needs no appeal to our tasks or our models. Once a chain diverges, being scored correct requires producing a value the model has never received and cannot derive; the probability of doing so by chance is one over the value space. This covers execution-match and AST-match benchmarks as they are ordinarily built. **Any work reporting a fitted self-correction, recovery, or severity parameter under such scoring is reporting a number its scoring rule determined in advance** -- and, as §4.4 records, we made exactly that mistake across three successive reworkings of this pipeline before seeing it.

The fix is small, and it is retrospective. Score a call against the state the model actually holds rather than the state it should have held. Concretely, that is one extra predicate per channel: for tool choice, "is this the tool the rule selects given the value held" -- which many routing-style evaluations already compute -- and for arguments, "is this the argument the rule requires of the value held". Both are functions of information the harness already has, because it knows what it fed the model.

The property that makes this adoptable rather than aspirational is that **it applies to data already collected**. Scoring is a pure function of the recorded completion and the recorded state, so any group holding raw completions can re-score without issuing a single new API call. We did exactly that: adding the predicate and re-scoring our existing cache cost 0 calls and 881 cache hits, and moved severity from a pinned 1.000 to +0.149 [+0.021, +0.268] and +0.316 [+0.066, +0.503]. Nothing was re-run.

And it changes what the numbers mean, not just their values. Under gold-agreement, all propagation loss is attributed to severity. Under conditional scoring, most of it turns out to be the gold trajectory becoming unreachable, with a smaller and separately estimable competence effect. Those are different quantities with different remedies: an agent that degrades under corruption needs better robustness, while an agent that merely cannot get back onto a trajectory needs a way to detect inconsistency and restart. Exact-match scoring cannot tell them apart, and reports the union as severity.

Our recommendation is to report both. Gold-agreement remains the right headline: it is comparable across systems and it answers the operational question of whether the task was done. Conditional scoring is a *requirement*, not an enhancement, for any claim about severity, recovery, or self-correction. The cost of the second is real -- there is no single right answer per call, cross-system comparison loosens, and a high correct-invocation rate no longer implies end-task success -- which is precisely why it should sit alongside the first rather than replace it.

The central methodological result of this work, independent of which real models are measured, is that a fit-free propagation-loss metric and a parametric severity/recovery model answer different questions and should be reported together, not as substitutes for one another. L_i answers "how much reliability is lost to propagation, and does that ordering hold up statistically" — a question exact-match scoring can answer cleanly. The parametric model answers "why," in terms of an interpretable severity-versus-recovery decomposition — a question that, on integer-chain synthetic tasks under exact-match scoring, current evidence suggests is not fully separable from the data alone. Whether this limitation is intrinsic to the *scoring regime* (exact match) or the *task design* (no graded closeness) is answerable directly from the real-benchmark comparison in Section 5.7, because BFCL/tau-bench arguments (city names, dates, quantities) do have graded closeness that our integer chains do not.

[Once real results are in: discuss (a) which models show the largest propagation gap and whether it tracks parameter count, family, or function-calling tuning; (b) whether the calling-mode ablation reveals a material fairness confound; (c) whether real-benchmark identifiability differs from the synthetic result, and what that implies for future tool-reliability evaluation design.]

Notes on completeness

Citations. Full citation verification is in progress; load-bearing citations — those where the text attributes a specific finding to a work rather than citing it as background — have been checked. `docs/CITATIONS_AUDIT.md` carries the per-citation checklist and is explicitly marked unverified for the remainder.

Adversarial review. A full adversarial review pass has not been completed. One is recommended before any resubmission or camera-ready deadline.

Data Collection Note

The final sample sizes in this paper are below the originally planned design, and the reason is external: every model was served on a provider free tier whose binding constraint is an undocumented per-model daily token allowance (Section 3.7). Routing-task cost is roughly quadratic in dependency depth, so the planned sweep required more days of allowance than the project had. We state the consequences once, plainly, and then report throughout with the actual n attached to every number.

What was collected. Five models reached usable sample sizes on the primary (copy-argument routing) arm, spanning depths 1, 2, 4 and 6, with one model also reaching depth 8. Per-model n is given in Table 1 and repeated beside every per-model claim. Which claims are full-power and which are directional is stated at each point rather than in aggregate.

What could not be collected.

- `groq/openai/gpt-oss-20b` reached **zero completed tasks** and is excluded from all per-model claims. It is retained in the paper as an attempted entry with its actual n stated, because the reason is itself a reproducibility finding: pilot runs earlier the same day consumed 199,895 of its 200,000 daily tokens, so it began the real sweep a full day's allowance behind and never recovered the gap within the window. A pilot spends the same allowance as the real run, invisibly, because the pilot costs minutes of wall clock and the cost is only felt in the next day's quota.
- The **transformed-argument condition** and the **randomised-presentation-order control** were designed, implemented, and verified feasible (Sections 5.4a, 5.4b), and were not executed. The claims that depended on them are reported as open rather than resolved.
- The **calling-mode ablation** was designed and verified feasible -- provider-native tool calling was confirmed working on five of six models by direct probe -- and was not executed.

What this costs the paper, stated precisely. The scoping caveat on the conditional-scoring decomposition (Section 5.4a) is *not* resolved: it would have been tested by the transformed condition, and remains a limitation on that result's generality. The parity/position-bias confound (Section 5.4b) is addressed by the discrimination statistic, which separates the two accounts on the data in hand, but not by the direct control that would have removed the confound outright. And no claim is made about how much of any model's measured unreliability is a calling-mode artifact.

None of the numbers reported here were extrapolated, imputed, or adjusted to compensate for a thin sample. Where n is small the interval is wide and is printed as such.

7. Limitations

A recurring methodological hazard, stated first because it is the most transferable item in this section. Three separate problems in this study had one shape: a quantity estimated from part of the suite, or assumed for convenience, applied across a suite whose members differ in exactly the respect that quantity depends on.

The clearest case is rule-following. Aggregated over the variable the routing rule conditions on, four of five models were indistinguishable -- overall first-listed rates of 0.481-0.497, consistent with no position bias at all. Split by that variable, the same data resolves from +1.000 to -0.182, and reveals that one model is *anti*-correlated with the rule rather than blind to it. **Discrimination is a per-model diagnostic of whether a model performs the task at all, distinct from how accurately it performs it, and it is invisible to any accuracy metric aggregated over the conditioning variable.** We give it first because it is the most externally validated of the three: the ordering it produces matches the reliability ordering established independently from L_t and error counts.

The other two: a single output-token cost (40 per call, measured on terse models) hid that one model costs 262.6 and needed 0.6 more days than projected; and measuring error composition per task depth hid that models differ in *where along a chain* they err, so a model with 0.857 of its errors on terminal steps read as immune to propagation. In each case the aggregate did not merely lose precision -- it pointed the wrong way, or concealed that a model was not performing the task.

The recommendation generalises beyond this study: **when evaluating a heterogeneous model suite, distrust every constant estimated from one model or assumed for convenience, and verify it per-model before trusting an aggregate built on it.** All three checks here were recomputations over data already collected.

- **Simulation-based validation cannot test channels the mock bypasses.** Three separate defects in this work were invisible to a validation suite that reported a healthy pipeline, and all three share one cause: the simulated policy received information through a channel the real interface does not have. It is called as `policy(task, step, ref, attempt)` and is *handed* the reference value it needs, whereas a real model is handed only a list of messages. The three: (i) the free-running loop appended neither the model's call nor the tool's result to the conversation, so a real model was asked for a value it had never seen, and measured g_t was exactly $1/\text{depth}$; (ii) the linear task family is degenerate on real models ($p_t = g_t = 1$, $L_t = 0$) because the tool order is announced and the argument is a verbatim copy, which no simulated policy revealed because policies err by construction; (iii) the simulated clean baseline was contaminated by cross-run-mode state, driving L_1 to -0.11 where it is 0 by construction. Two generalisable rules follow. A mock that receives state out-of-band cannot validate the channel that state is supposed to travel through. And tests must assert on the artifact sent to the provider rather than on the backend's behaviour, because behavioural assertions inherit the mock's blind spots by construction. We report this as a limitation of the methodology rather than as three incidents, because the pattern -- not any individual bug -- is what a replicator needs to guard against.
- **Withdrawn hypotheses and unimplemented analyses.** H5a and H5b concerned the error-feedback manipulation. Both feedback modes are implemented, but only the structured condition was run: exercising the manipulation requires repeating the full sweep, which the free-tier token budget cannot absorb. They are withdrawn rather than left listed as though tested. Likewise, an earlier plan specified a mixed-effects logistic model of per-call outcomes as a robustness check and as the vehicle for the calling-mode covariate; it was never built, so the calling-mode residual is reported as a direct contrast and the claim is removed. Release-date version pinning was also specified but is not possible: the providers expose no version field, and every model identifier in the original configuration had been retired by run time, so reproduction rests on the recorded identifier, the run date and the response cache.
- **Vacuous soft scoring on the synthetic suite.** Every synthetic argument is a single integer, so the soft (tolerance/normalised) score cannot differ from the strict score: a relative tolerance of 10^{-9} never separates two distinct integers, and there are no free-text or enum arguments for a normalised match to act on. The synthetic results therefore report the strict score alone. We did not add graded arguments to make the metric non-degenerate, since that would change the task design for the sake of a measurement; it is deferred to the real-benchmark stage, where graded closeness exists naturally and where it is also the direct test of whether π becomes better identified.
- **Distractor pressure differs between task arms.** The linear generator adds never-correct distractor tools; the routing generator exposes only branch tools, all reachable on some branch, and silently ignores the distractor parameter. No distractor sweep was run. Cross-arm inferences that depend on distractor pressure are therefore not drawn. Since linear serves only as a null control, and a control carrying *more* distraction that still yields $L_t = 0$ is the conservative direction for that argument, we tolerate the asymmetry rather than remove it.

- **Exact-match identifiability.** Section 4.3 establishes, on a fully controlled suite where ground truth is known, that the parametric severity/ recovery model is not cleanly identifiable under exact-match scoring. This should be treated as a property of the measurement regime unless and until the real-benchmark comparison (Section 5.7) shows otherwise.
- **Synthetic task realism.** The integer/modulo synthetic tasks give exact ground truth and contamination control, but have no graded notion of "almost correct," and their tool descriptions, while designed with paraphrase and distractor variation, remain more abstract than real-world API schemas. The real-benchmark comparison is the direct test of how much this matters.
- **Cost-bounded model coverage.** The model suite (Section 3.6) is chosen to isolate specific factors (scale, family, tuning) rather than to be exhaustive; conclusions about e.g. scale trends should be read as evidence from the sampled models, not as a claim about all models at that scale.
- **Provider and calling-mode heterogeneity.** Despite the uniform-protocol control and native-vs-uniform ablation (Section 5.6), some residual provider-specific effects on argument formatting may remain uncontrolled.
- **Recovery-model persistence assumption.** The three-state model assumes a poisoned context stays poisoned (of the same origin) until an explicit recovery event; because the synthetic tasks have full state labels, the rate at which real trajectories violate this assumption is measurable and should be reported as a check on the model's applicability.
- **Single-turn user simulation.** [If applicable given final real-benchmark choice: note whether tasks involve simulated multi-turn user interaction or only agent-tool interaction, and what that excludes.]

8. Conclusion

We presented an invocation-level framework for measuring tool-use reliability in language model agents, disaggregating tool selection from argument correctness and separating context-length degradation from error propagation via matched clean and free-running measurement. A fully validated pipeline, tested first on a controlled simulated suite with known ground truth, surfaced both real implementation bugs and a genuine, load-bearing methodological finding about the identifiability of parametric propagation models under exact-match scoring — a finding that survived every fix we made and that we carry forward as an explicit reporting discipline for real models. On [N] real models spanning [axes], we find [headline results]. We release the full pipeline, task suite, and analysis code to support reproducible, invocation-level evaluation of tool-use reliability going forward.

Appendix A: Reproducibility artifact

Code: [github.com/\[org\]/tool-use-reliability](https://github.com/[org]/tool-use-reliability). Key entry points: `scripts/estimate_cost.py` (budget before running), `scripts/run_real_suite.py` (the real experiment), `scripts/run_full_analysis.py` (the simulated validation in Section 4), `tests/test_regression.py` (locks in the scoring fixes from Section 4.2). Model list, task parameters, and feedback/calling modes are set in `config/default.yaml`.

Appendix B: Simulated-suite configuration

model	p0	pi	syntax_share	r_syn	r_sem
fam0-weak	0.78	0.70	0.50	0.35	0.05
fam0-mid	0.88	0.45	0.50	0.55	0.15
fam0-strong	0.95	0.22	0.50	0.75	0.30
fam1-weak	0.75	0.65	0.35	0.30	0.05
fam1-mid	0.86	0.40	0.35	0.50	0.12
fam1-strong	0.94	0.18	0.35	0.70	0.25

`p0` is the clean baseline at depth 1; each policy's baseline declines linearly with depth (`p_slope`, see `src/tur/harness/sim_policy.py`). All other parameters are as defined in Section 3.2.

References

- Chen et al. (2025). ACEBench: A comprehensive evaluation of tool usage. Findings of EMNLP.
- Guo et al. (2024). StableToolBench: Towards stable large-scale benchmarking of tool learning. Findings of ACL.
- Healy et al. (2026). Internal representations as indicators of hallucinations in agent tool selection.
- Li et al. (2023). API-Bank: A comprehensive benchmark for tool-augmented LLMs. EMNLP.
- Lin et al. (2024). Hammer: Robust function-calling for on-device language models via function masking.
- Liu et al. (2025). ToolACE: Winning the points of LLM function calling. ICLR.
- Maekawa et al. (2025). Towards reliable benchmarking: A contamination-free, controllable evaluation framework for multi-step LLM function calling (FuncBenchGen).
- Patil et al. (2023). Gorilla: Large language model connected with massive APIs.
- Patil, Yan et al. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation. ICML.
- Qin et al. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. ICLR.
- Schick et al. (2023). Toolformer: Language models can teach themselves to use tools. NeurIPS.
- Shinn et al. (2023). Reflexion: Language agents with verbal reinforcement learning. NeurIPS.
- Wang et al. (2025). MTU-Bench: A multi-granularity tool-use benchmark for large language models. ICLR.
- Xu et al. (2025). Reducing tool hallucination via reliability alignment. ICML.
- Yao et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR.
- Yao et al. (2024). tau-bench: A benchmark for tool-agent-user interaction in real-world domains. ICLR 2025.
- Ye et al. (2024). RoTBench: A multi-level benchmark for evaluating the robustness of large language models in tool learning. EMNLP.

⚠ Citation verification still pending (carried over from the literature review): confirm every citation above — exact author lists, venues, and years — before this reference list is submission-ready. Several are recent preprints (2025–2026) that may not have final published forms yet.